

Subida de ficheros a shell

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando una app permite subir ficheros, el riesgo es doble: que acepte un fichero con código ejecutable (un .php, .aspx, .jsp) donde el servidor interpreta ese lenguaje, Y que ese fichero acabe en una ruta accesible por HTTP donde puedas pedirlo. Si ambas cosas se cumplen, subir un fichero se convierte en ejecutar código.

El programador valida de forma incompleta: solo la extensión (pero acepta .phtml o doble extensión), solo el Content-Type (que tú controlas y puedes falsear), o solo el nombre (pero no el contenido real, los 'magic bytes'). Cualquier validación que dependa de datos que TÚ envías es evitable.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier formulario de subida: avatar, adjuntos, importación de ficheros, subida de temas/plugins (WordPress). Prioriza probarlo si la app es de un stack que ejecuta ficheros interpretados (PHP es el caso más común en CTF).

Cómo identificarla

- El formulario solo comprueba la extensión del NOMBRE, no el contenido real del fichero.
- Puedes predecir o descubrir la ruta donde se guardan los uploads (uploads/, /files/, patrón de nombre).
- El Content-Type que TÚ envías en la petición es el que decide la validación (fácilmente falseable).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sube un fichero con extensión ejecutable disfrazado de imagen (Content-Type image/png) y comprueba si el servidor lo acepta y donde lo guarda.

Evidencia esperada: El fichero se sube sin rechazo y puedes localizarlo (ruta predecible o listado); al pedirlo por HTTP, el servidor lo EJECUTA en vez de servirlo como archivo estático.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS -F 'file=@shell.phtml;type=image/png' "$URL/upload"
```

Objetivo y contexto: Sube un fichero .phtml (extension alternativa que muchos servidores PHP tambien ejecutan) declarando un Content-Type de imagen para intentar pasar validaciones basadas en ese campo.

Resultado esperado: La respuesta suele indicar exito y a veces la ruta del fichero subido. Si no la da, prueba rutas predecibles (uploads/shell.phtml) o busca un listado de directorio.

Anatomía de los payloads

La primera prueba es `curl -sS -F 'file=@shell.phtml;type=image/png' "$URL/upload"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El fichero se sube sin rechazo y puedes localizarlo (ruta predecible o listado); al pedirlo por HTTP, el servidor lo EJECUTA en vez de servirlo como archivo estatico.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prueba primero un upload normal para entender el flujo: donde se guarda, como se nombra, si redirige a una URL del fichero.
- Prueba bypasses de extension: doble extension (shell.php.jpg), extensiones alternativas que el servidor tambien ejecuta (.phtml, .phar), o null byte si el stack es muy viejo.
- Si valida Content-Type, cambialo a image/png o similar mientras el contenido sigue siendo tu shell.
- Localiza el fichero subido y pidelo por HTTP; si se ejecuta, confirma con un payload minimo antes de una reverse shell completa.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor valida magic bytes Y extension Y Content-Type de forma consistente, y ademas sirve los uploads desde una ruta sin permiso de ejecucion.
- Los ficheros subidos se guardan fuera del webroot o se renombran de forma no predecible sin que puedas listarlos.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un formulario de subida acepta un fichero que el servidor luego EJECUTA; súbelo, encuentra su ruta, y ejecútalo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (file-upload)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.